



PART 3 — USER INTERFACE

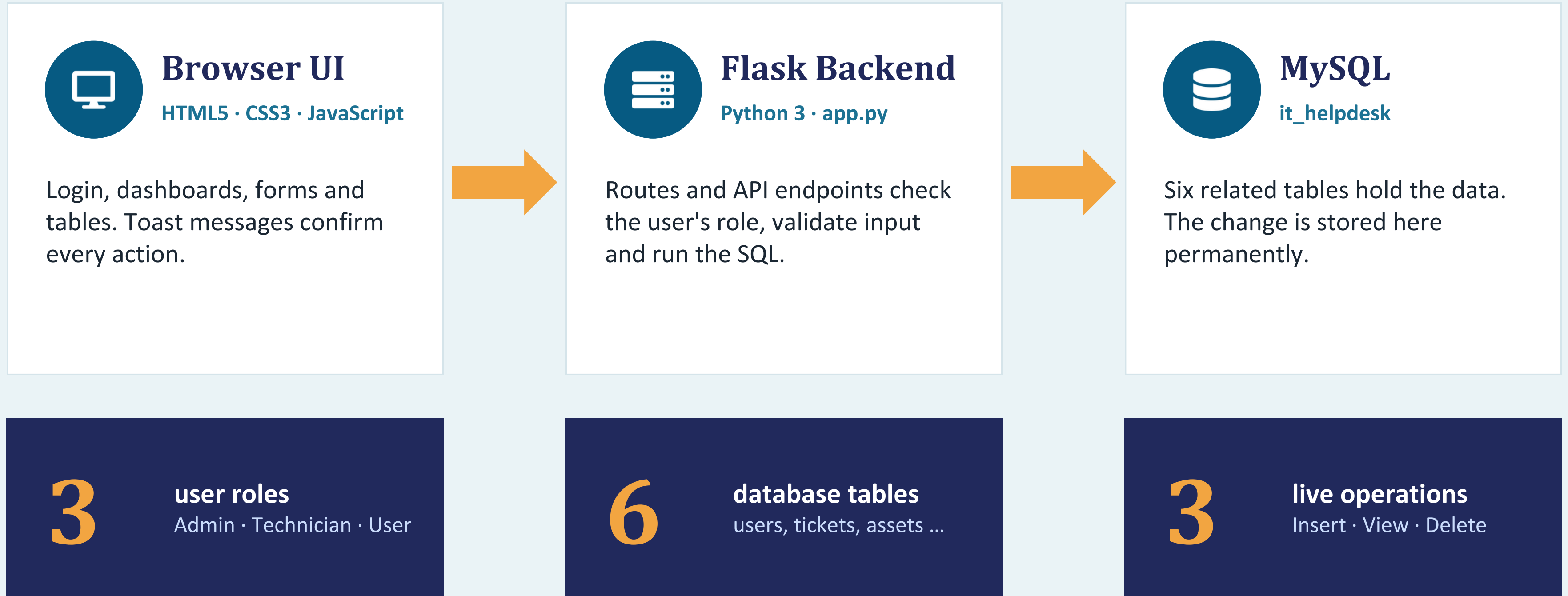
A Working Web UI on MySQL for the IT Helpdesk & Asset Support Management System

[Insert Records](#) • [Delete Records](#) • [View Records](#) • [Live Demonstration](#)

Flask web application connected to a live MySQL database

How the UI Connects to MySQL

Every button on the screen becomes a request that ends as a SQL statement



Three Operations to Demonstrate

Each operation is shown live, then confirmed directly in the database



Insert

Raise a new ticket or add an asset using a form

SQL STATEMENT

INSERT INTO

Available to: Admin · User



View

Browse tickets and assets in tables with joined details

SQL STATEMENT

SELECT ... JOIN

Available to: Admin · Technician · User



Delete

Remove a record after a confirmation prompt

SQL STATEMENT

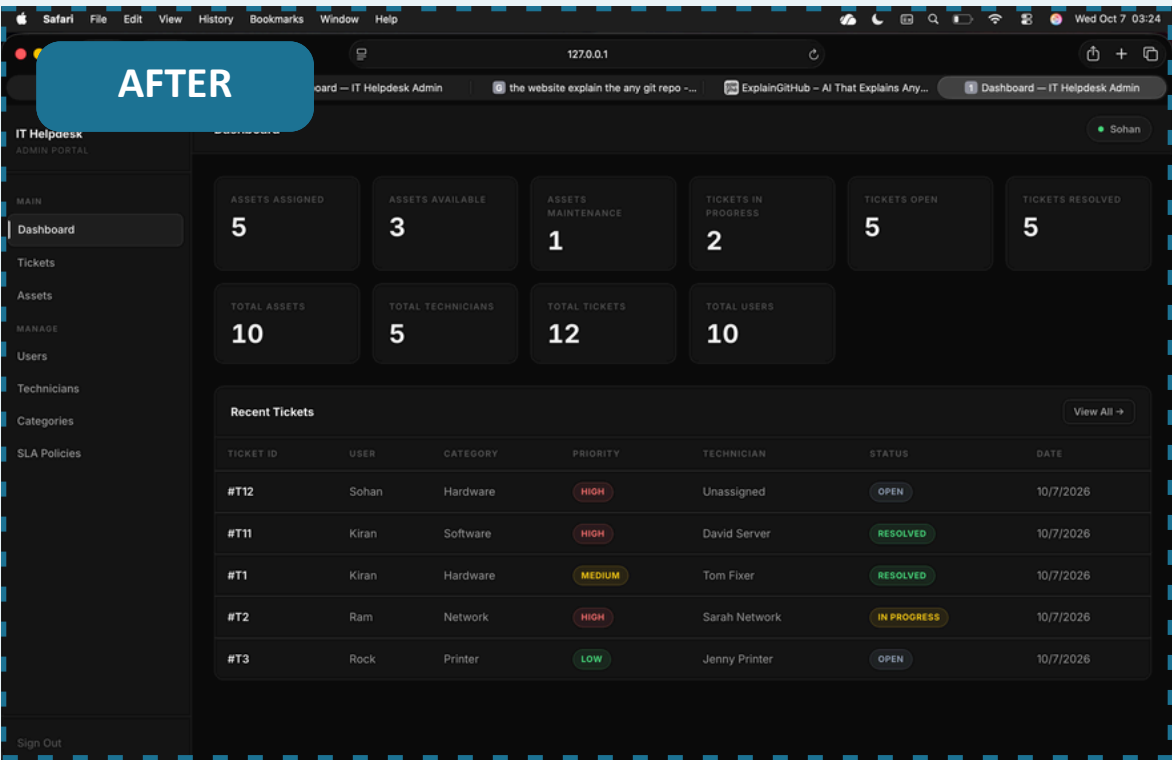
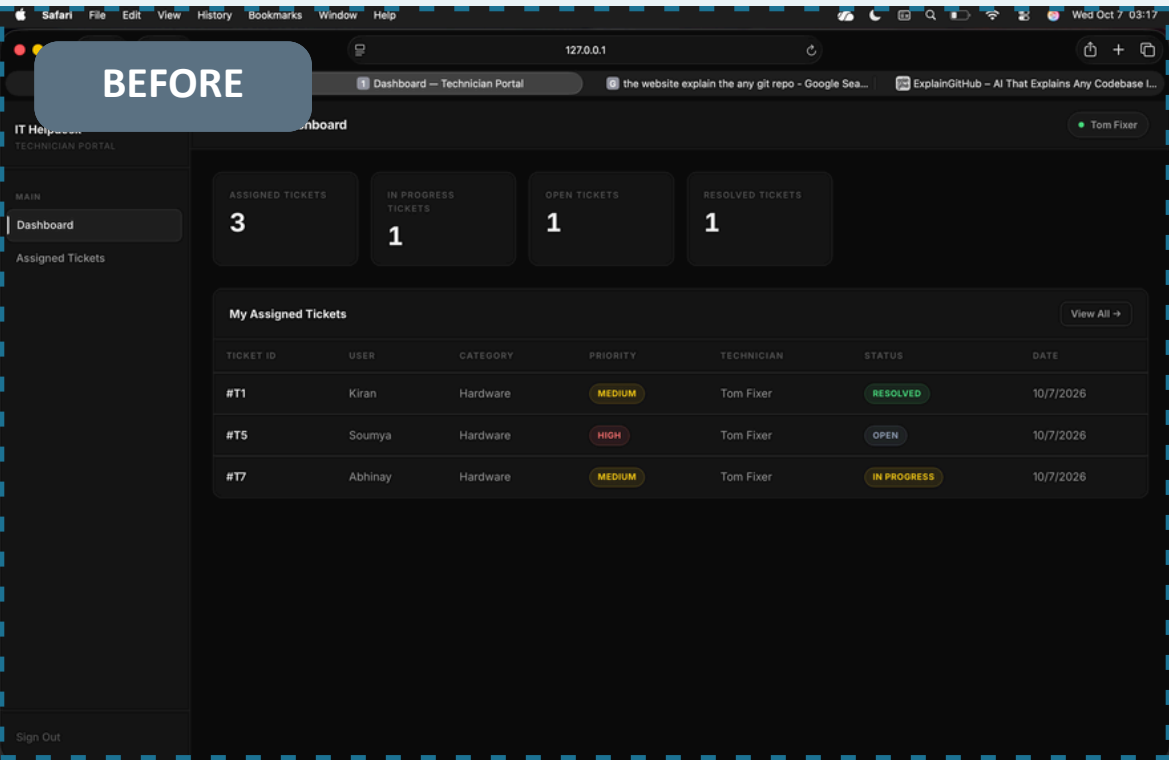
DELETE FROM

Available to: Admin

Proof: after each action we run a SELECT in MySQL to show the change is really stored.

Insert — Raising a New Ticket

A form submission becomes a new row in the tickets table



What happens

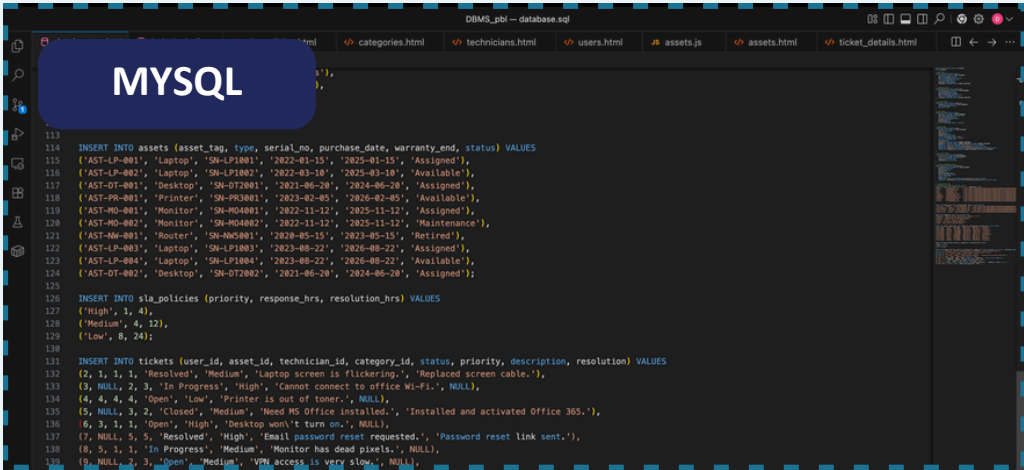
- User opens the New Ticket form
- Picks category, asset and priority
- Flask runs the INSERT and returns a toast
- New ticket appears in the list

```

INSERT INTO tickets (user_id, asset_id, category_id, priority, status)
VALUES (12, 101, 1, 'High', 'Open');

Query OK, 1 row affected
SELECT * FROM tickets ORDER BY ticket_id DESC LIMIT 1;

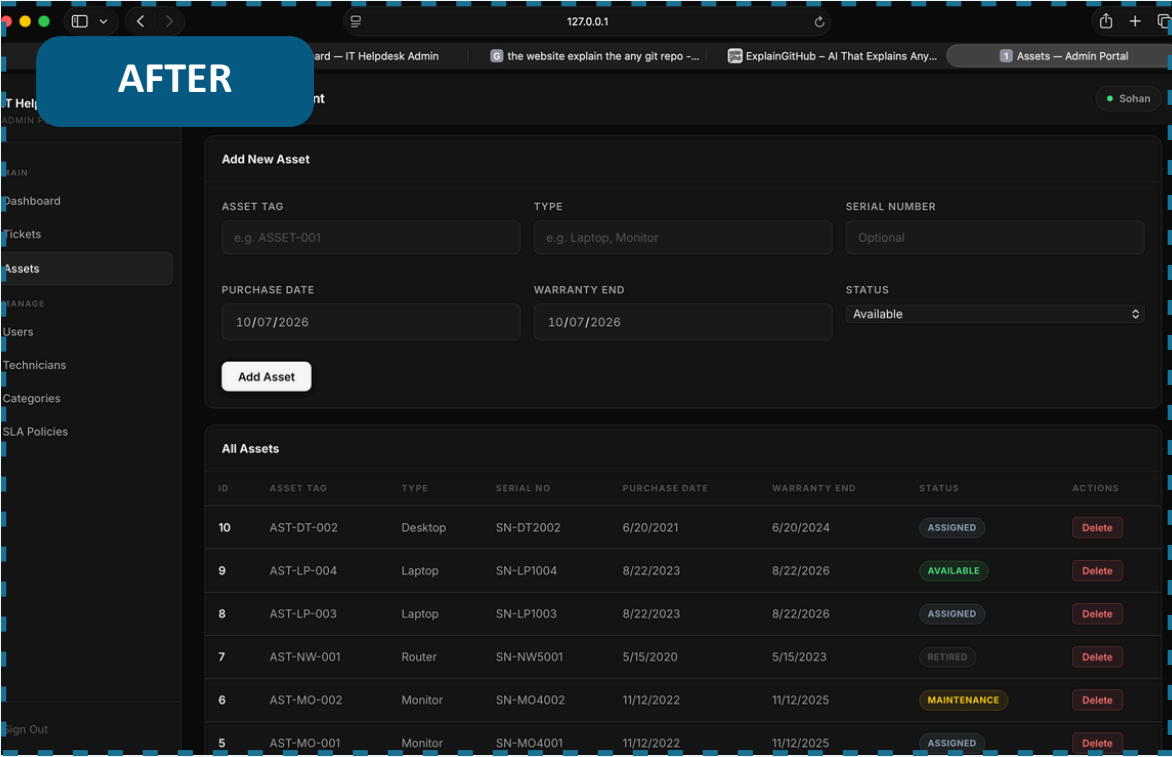
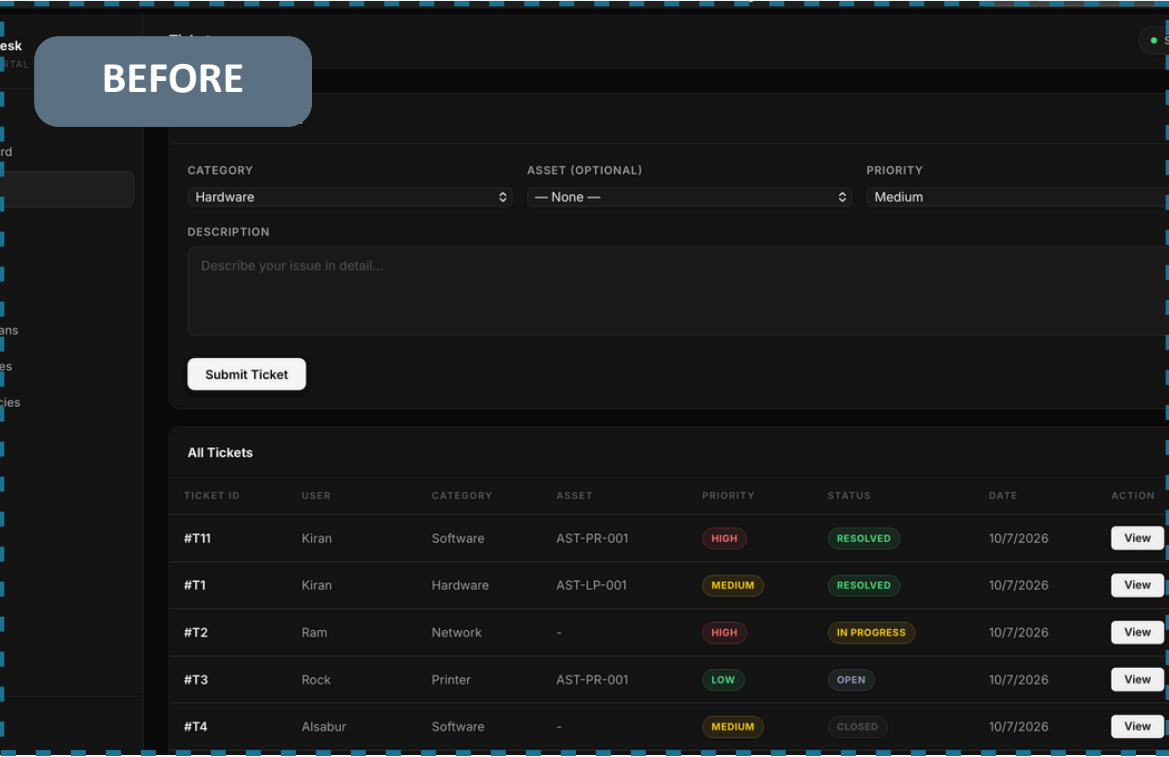
```



Check: the row count in tickets goes up by one and the new ticket_id is visible in MySQL.

View — Browsing Records

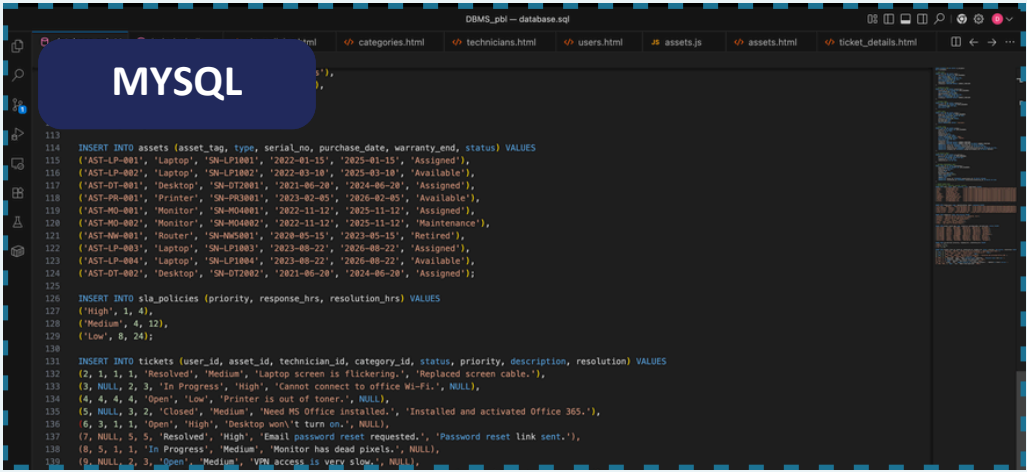
Joined queries turn foreign keys into readable tables



What happens

- Open the Tickets or Assets page
- Flask runs a SELECT with joins
- Results render as a table with user, asset and technician names
- Technicians and users see only their own tickets

```
SELECT t.ticket_id, u.name, a.asset_tag, c.name, t.priority, t.status
FROM tickets t
JOIN users u ON t.user_id = u.user_id
JOIN assets a ON t.asset_id = a.asset_id
JOIN categories c ON t.category_id = c.category_id;
```



Check: the table on screen matches the rows returned by the query in MySQL.

Delete — Removing a Record

A confirmed action removes the row from the database



```
DELETE FROM assets WHERE asset_id = 103;
```

```
Query OK, 1 row affected
```

```
SELECT COUNT(*) FROM assets WHERE asset_id = 103;
```

```
0
```



What happens

- Click Delete on a record
- A confirmation dialog prevents accidents
- Flask runs the DELETE
- The row disappears from the table and from MySQL

Check: the deleted record no longer appears in MySQL, and foreign keys keep related tables consistent.

Live Demonstration Flow

The order we will follow during the presentation



1

Log in

Sign in with a demo role account

2

Insert

Submit the new ticket form

3

Verify

Run SELECT in MySQL to see the new row

4

View

Open the list and find the record

5

Delete

Confirm the deletion prompt

6

Verify

Run SELECT again: row is gone

Result: every change made in the UI is visible in MySQL straight away, with no manual refresh of the database.

Built-in Safeguards

Features that keep the data safe while users insert and delete



Role-based access

Admin, Technician and User portals; session check on every protected route.



Hashed passwords

Werkzeug script hashing, so no plain-text passwords are stored.



Delete confirmation

A dialog asks before a record is removed, to prevent mistakes.



Instant feedback

Toast notifications confirm success or show an error after each action.



Referential integrity

Foreign keys in tickets link users, technicians, assets and categories.



Filtered views

Each role sees only the tickets and data it is allowed to see.

Conclusion & Submission

A working UI connected to MySQL supports insert, delete and view, with each change shown in the database.



PPT / PDF

This presentation



Source code

app.py, templates, static files,
database.sql



Screenshots

Insert, delete and view screens,
before and after

Uploaded to Presentation-III

Thank you